An Empirical Study of Issues in GPU Kernel Libraries for LLM Systems

Yanjie Gao[1], Jiayi Bi[2], Haoxiang Lin[1], Ling Feng[2]
[1]Microsoft Research Asia, Beijing, China [2]Tsinghua University, Beijing, China
Email: {yanjga,haoxlin}@microsoft.com, {bijy22,fengling}@mails.tsinghua.edu.cn

arXiv:2611.09280v1 [cs.AI] 6 Nov 2026

***Abstract*—Behind the expanding use of large language models (LLMs) across application domains lies a growing dependence on specialized GPU kernel libraries. These libraries underpin the correctness and efficiency of training and inference across diverse GPU platforms, implementing attention, GEMM, quantization, and low-precision operators while managing tensor shapes, memory layouts, code generation, and hardware-specific dispatch. Problems at this layer can silently corrupt outputs, hang GPUs, prevent compilation, introduce deployment incompatibilities, or degrade performance, thereby reducing developer productivity and wasting expensive GPU resources.**

We present the first comprehensive empirical study of issues in GPU kernel libraries for LLM systems. We manually analyzed 300 high-quality GitHub issue reports, their discussions, and available fixes from seven representative libraries: FlashInfer, FlashAttention, Triton, CUTLASS, DeepGEMM, xFormers, and bitsandbytes. The analysis characterizes symptoms, root causes, fixes, and testing and debugging practices. Our main findings are: (1) 60.33% of the sampled issues originate in the Compiler and Build System or Runtime Environment dimensions, so validation must cover the deployment and execution stack, not just kernel code. (2) Kernel specialization imposes implicit correctness constraints on tensor shapes, metadata layouts, memory aliasing, and hardware resources. Violations can produce incorrect outputs or launch failures even for supported operations, highlighting the need for explicit contract validation and boundary-case testing. (3) Current testing and debugging practices are often insufficient for identifying low-precision numerical problems and hardware-specific execution failures. We outline research opportunities and tooling directions to support future LLM GPU kernel development.

***Index Terms*—Empirical studies, GitHub issues, GPU kernel failures, LLM systems.**

I. Introduction

In recent years, large language models (LLMs) have become a core component of modern software systems. Their applications range from chatbots and programming assistants to search, education, and enterprise software. Training these models and serving their requests require large-scale GPU infrastructure, where tensor operations account for much of the runtime cost.

Developers rely on specialized GPU kernel libraries, such as FlashAttention, Triton, and CUTLASS, to execute these workloads efficiently [1]–[7]. These libraries provide optimized implementations of attention, GEMM, and quantization. Their implementations incorporate low-precision arithmetic and hardware-specific code generation to support diverse tensor shapes and layouts, data types, and GPU architectures.

Failures in these libraries can cause crashes, hangs, incorrect outputs, deployment failures, and performance regressions, reducing developer productivity and wasting expensive GPU resources. For example, a GPU kernel that produces incorrect results may silently corrupt generated tokens without raising a runtime exception; a synchronization bug may hang a long-running inference job, while a packaging or CUDA incompatibility may prevent deployment before a kernel is even tested. Worse still, these failures' impacts can extend further than those of failures in ordinary application code because a single kernel path may be reused across many models, serving requests, GPUs, and downstream systems.

The cause of such a failure is often difficult to isolate. Its visible symptom may depend on a particular combination of tensor shape/layout, data type, GPU architecture, CUDA or driver version, generated code path, and downstream framework. These challenges underscore the need for a systematic understanding of GPU kernel failures and their underlying causes, which can inform efforts to prevent, detect, debug, and resolve such failures. As LLMs become integral to modern software, the reliability of the GPU kernel libraries supporting their training and inference deserves a high degree of attention from software engineering researchers and practitioners.

Prior empirical research has examined problems in machine-learning and deep-learning software from different perspectives. Shen et al. [8] characterized defects in deep-learning compilers, while Humbatova et al. [9] developed a fault taxonomy for deep-learning applications. Gao et al. [10] investigated quality issues in a deep-learning platform. More recently, a study of LLM training systems highlighted failures involving parallel execution, configuration, and external dependencies [11]. These studies explain many problems in applications, frameworks, compilers, and end-to-end training systems, but the GPU kernel library layer remains largely unexplored. Diagnosis at this layer requires reasoning about tensor shapes and layouts, vectorized memory access, synchronization, low-precision semantics, code generation, and architecture-specific kernel dispatch.

This paper presents the first comprehensive empirical study of issues in GPU kernel libraries for LLM systems. We manually analyzed 300 high-quality GitHub issue reports from seven representative libraries: FlashInfer [2], FlashAttention [1], Triton [3], CUTLASS [4], DeepGEMM [5], xFormers [6], and bitsandbytes [7]. Our analysis draws on issue titles, descriptions, discussions, diagnostic evidence, and available fixes. We classified the reports into 6 symptom categories and identified 15 root-cause categories, which were further organized into 3 dimensions. We also examined typical fixes

and the testing and debugging practices documented in the reports. The goal is to provide a systematic account of failures in GPU kernel libraries that can inform the development of more reliable and efficient LLM systems.

Specifically, this study addresses three research questions:

RQ1: What are the common symptoms of issues in GPU kernel libraries for LLM systems? We examine the titles and initial descriptions of the 300 issue reports to identify their observable symptoms. The issues fall into 6 categories: Deployment Failure (27.33%), Build Failure (24.33%), Crash (21.00%), Incorrect Functionality (17.67%), Performance Degradation (7.67%), and Hang (2.00%). Section IV presents the symptom analysis.

RQ2: What causes these issues, and how are they typically fixed? Issue discussions, code fixes, and maintainer conclusions provide the evidence for identifying root causes and summarizing typical fixes. We identify 15 root-cause categories across 3 dimensions: Kernel Library (39.67%), Runtime Environment (33.33%), and Compiler and Build System (27.00%). These causes often reflect rapid changes in hardware and software, complex compiler and package dependencies, and specialization-specific assumptions about tensor shapes and layouts, data types, and execution resources. The root causes and corresponding fixes are detailed in Section V.

RQ3: What testing and debugging practices are commonly used? We review issue discussions, reproduction scripts, diagnostic output, and relevant reference materials to examine how developers test and debug GPU kernels. The analysis reveals that current practices are often insufficient for diagnosing low-precision numerical errors and hardware-specific execution failures. Section VI summarizes these practices and their limitations.

The findings point to research opportunities and tooling improvements for GPU kernel libraries used by LLM systems. We discuss their implications alongside the corresponding findings in Section VII.

In summary, this paper makes the following contributions:

1) We present the first comprehensive empirical study of issues in GPU kernel libraries for LLM systems. Based on 300 high-quality issue reports from seven representative libraries, the study characterizes symptoms, root causes, typical fixes, and testing and debugging practices.
2) We draw practical implications from the findings and suggest improvements to GPU kernel libraries. These suggestions aim to improve developer productivity and the reliability and efficiency of LLM systems.

II. Background

Large language model (LLM) applications rely on training and inference systems that execute large tensor programs on GPUs. Python is commonly used for model training, evaluation, and serving because it offers flexible APIs and a broad ecosystem. The performance-critical operators, however, are handled by specialized libraries that supply handwritten GPU kernels, compiler-generated kernels, and reusable CUDA templates. Examples include FlashAttention [1], FlashInfer [2], Triton [3], CUTLASS [4], DeepGEMM [5], xFormers [6], and bitsandbytes [7]. This division allows LLM systems to offer high-level abstractions while specialized GPU kernels meet their latency, throughput, memory capacity, and numerical-precision requirements on modern GPUs.

```
1  def attention(q, k, v, causal):
2      # 1. Check the operator contract.
3      check_shape_stride_dtype(q, k, v)
4      # 2. Select a kernel for the input and platform.
5      target = get_gpu_architecture()
6      config = (q.dtype, q.shape[-1], causal, target)
7      kernel = dispatch_table.lookup(config)
8      # 3. Build a specialization when needed.
9      if not kernel.is_prebuilt:
10         kernel = jit_compile(kernel, config)
11     # 4. Launch and return the device result.
12     return kernel.launch(q, k, v)
```

Fig. 1. Simplified workflow for an attention operator within a GPU kernel library: input validation, kernel selection, compilation, and execution.

GPU kernel libraries sit between model operators and the hardware that executes them. Attention libraries optimize prefill, decode, sliding-window attention, paged KV-cache layouts, causal masking, and variable sequence lengths. GEMM libraries and domain-specific languages (DSLs) provide access to tensor-core instructions, tiling strategies, shared-memory layouts, TMA transfers, warpgroup MMA, and low-precision formats. Quantization libraries support weight-only and activation-aware schemes in formats such as INT8, INT4, FP8, and FP4.

A framework or serving engine may invoke these libraries indirectly during execution. Which kernel actually runs depends on the tensor shape, layout, and data type, as well as the CUDA version and GPU architecture. The selected path may use handwritten CUDA code, generated Triton code, CuTe templates, or packaged binaries. Users may work entirely through high-level Python APIs, yet their programs remain subject to the contracts and failure modes of the underlying kernel libraries.

Before an operator can run, its kernel library must be installed or built for the framework and environment. Figure 1 illustrates the subsequent library-internal workflow with a simplified attention example. The library first validates the input tensors (line 3). Next, it determines the GPU architecture and constructs a dispatch configuration from the data type, head dimension, causal mode, and target architecture (lines 5–7). If no prebuilt version of the selected implementation is available, the library compiles a specialization (lines 9–10). The selected kernel is then launched, and its result is returned (line 12).

An issue in a GPU kernel library can arise at any of these stages. Package or binary incompatibilities can prevent installation. Validation may fail to enforce an input contract, and dispatch may select an incorrect implementation or find no suitable one. JIT compilation may fail before execution begins, while device execution may crash, produce incorrect results, or run inefficiently. These library-level responsibilities distinguish GPU kernel libraries from complete systems such as Megatron-

LM [12], vLLM [13], and SGLang [14]. Those systems additionally handle model loading, scheduling, distributed execution, and service-level logic.

III. Methodology

A. Study Subjects

LLM development and deployment increasingly rely on GPU kernel libraries, whose specialized implementations pose challenges in correctness, compatibility, and efficiency. Problems in these libraries can propagate across downstream training and inference systems, wasting GPU resources and reducing developer productivity. Examining reported issues is therefore critical for improving the quality and reliability of LLM training and inference.

GitHub issues from GPU kernel library repositories provide the empirical basis for our study. To the best of our knowledge, prior empirical studies [8], [10], [11], [15]–[17] have not specifically investigated issues in GPU kernel libraries for LLM systems. Our objectives are to characterize the issues' symptoms, identify their root causes, analyze the corresponding fixes, and examine testing and debugging practices. The study covers seven representative open-source repositories. Among them, FlashAttention, FlashInfer, and xFormers provide attention and LLM operator kernels, while CUTLASS and DeepGEMM provide GEMM and low-precision matrix kernels. The remaining two repositories serve distinct roles: Triton provides GPU compilation and code generation, and bitsandbytes provides quantization kernels and CUDA packaging. Together, these projects span complementary parts of the LLM GPU kernel ecosystem. Complete serving engines fall outside our scope because their issue trackers also encompass model-loading, scheduling, distributed-runtime, and service-level problems, which are not within the scope of this study.

These libraries are extensively used in both industry and academia. Research on efficient attention and GPU programming gave rise to FlashAttention and Triton, respectively. Meanwhile, CUTLASS, FlashInfer, DeepGEMM, xFormers, and bitsandbytes offer reusable CUDA templates, inference kernels, low-precision GEMM, attention operators, and quantization kernels. Their integration into downstream systems further illustrates their practical relevance. For training, Megatron-LM, PyTorch [18], and Transformers [19] use or expose integrations with optimized attention, Triton/CUDA kernels, xFormers operators, and bitsandbytes quantization. For inference, vLLM integrates FlashAttention, FlashInfer, xFormers, Triton, and bitsandbytes backends, whereas SGLang integrates FlashInfer, FlashAttention, Triton, and DeepGEMM backends.

B. Data Collection

GitHub is a widely used source for empirical investigations of software issues. Following prior research [20], we collected reports from the official repositories of the selected libraries to examine problems encountered during their development and use. These repositories remain active, with maintainers continually refining GPU kernel implementations, compiler support, and platform compatibility. Users contribute not only failure reports but also reproduction details, diagnostic information, and input data to the resolution process. Compared with commits, issue reports provide a richer account of a problem, spanning its initial description, subsequent discussion, diagnostic evidence, and developer resolution [20]. These contexts make GitHub issues particularly suitable for examining symptoms, identifying root causes, and analyzing fixes.

TABLE I
Dataset by repository.

Repository	Raw	Candidate	Final
FlashInfer	1,051	390	37
FlashAttention	1,786	302	28
Triton	2,205	815	77
CUTLASS	1,931	597	56
DeepGEMM	180	38	4
xFormers	722	206	19
bitsandbytes	1,224	833	79
Total	9,099	3,181	300

Using the GitHub REST API [21], we collected all open and closed issues from the seven repositories between December 26, 2015 and July 10, 2026. For each report, we downloaded its title, description, labels, state, timestamps, comments, and linked GitHub references. The initial collection comprised 9,099 issues. To focus on concrete and complete failures of existing functionality, we first removed open issues and retained only closed reports meeting this criterion. Repository-specific labels, considered alongside report content, informed the filtering process. Screening began by locating issues whose titles contained explicit failure-related terms, including "bug", "error", "fail", "wrong", "crash", "hang", "regression", "performance", and "numerical issue". We further eliminated placeholders, support questions, feature or capability requests, documentation and maintenance tasks, broad tracking issues, reports lacking explicit failure evidence, and duplicates. Labels such as "question", "feature request", "enhancement", "documentation/docs", "roadmap", "tracking", "help wanted", "good first issue", "chore", and "style" helped identify reports outside the study scope. The screening process yielded a candidate issue pool of 3,181 closed reports.

From this pool, we finally constructed a repository-stratified sample of 300 issues, with each repository's sample size proportional to its number of candidate reports. Each sampled issue was then examined alongside its discussion, diagnostic evidence, and available fix. Reports containing only symptoms, speculative explanations, or competing explanations were excluded. Each excluded report was replaced with an issue from the same repository whose creation time was closest to that of the excluded report. This screening-and-replacement process continued until the final dataset contained 300 high-quality issues. Table I summarizes the raw collection, candidate dataset, and final sample for each repository.

C. Data Labeling

Symptom labeling focused on observable outcomes, drawing on each issue's title, the opening portion of its description, error messages, and reported behavior. We adapted terminology

TABLE II
CLASSIFICATION OF COMMON SYMPTOMS.

Category	Total		FlashInfer		FlashAttention		Triton		CUTLASS		DeepGEMM		xFormers		bitsandbytes	
	No.	Ratio	No.	Ratio	No.	Ratio	No.	Ratio	No.	Ratio	No.	Ratio	No.	Ratio	No.	Ratio
Deployment Failure	82	27.33%	3	1.00%	5	1.67%	13	4.33%	6	2.00%	0	0.00%	8	2.67%	47	15.67%
Build Failure	73	24.33%	7	2.33%	5	1.67%	29	9.67%	22	7.33%	2	0.67%	3	1.00%	5	1.67%
Crash	63	21.00%	18	6.00%	6	2.00%	10	3.33%	8	2.67%	1	0.33%	4	1.33%	16	5.33%
Incorrect Functionality	53	17.67%	6	2.00%	7	2.33%	16	5.33%	13	4.33%	1	0.33%	1	0.33%	9	3.00%
Performance Degradation	23	7.67%	2	0.67%	4	1.33%	7	2.33%	5	1.67%	0	0.00%	3	1.00%	2	0.67%
Hang	6	2.00%	1	0.33%	1	0.33%	2	0.67%	2	0.67%	0	0.00%	0	0.00%	0	0.00%
Total	300	100.00%	37	12.33%	28	9.33%	77	25.67%	56	18.67%	4	1.33%	19	6.33%	79	26.33%

and categories from prior empirical studies [8], [10], [11], [16], [17], [22]–[26] to the context of GPU kernel libraries. Each issue received one primary symptom label from "Build Failure" [22], [23], [26], "Deployment Failure" [10], [27], "Crash" [8], [10], [11], [22], "Incorrect Functionality" [22], [23], [26], "Performance Degradation" [22], [25], or "Hang" [11], [28]. Hardware, data type, trigger, downstream context, and diagnostic evidence were also recorded as auxiliary information. Table II presents the 6 resulting symptom categories.

Root-cause and fix analysis followed the resolution process of each issue, including its discussion, maintainer diagnosis, reproduction results, code analysis, and available repair. Building on terminology from prior studies of deep-learning-related issues and bugs [8], [10], [11], [23], [26], we adapted the categories according to the mechanisms observed in the selected reports. Each issue was assigned a single root-cause category, for example, "Concurrency Error" [8], [11], "API Misuse" [11], [23], [26], "Incorrect Numerical Computation" [8], or "Software Incompatibility" [8], [11]. Domain knowledge and patterns observed in the reports also guided our classification of root causes not covered by the existing taxonomy, including "Improper Kernel Dispatch" and "Incorrect Compiler Transformation", which yielded further insights into GPU kernel library issues. Finally, we organized the 15 identified root-cause categories into 3 dimensions to provide a comprehensive view. Table III presents the overall taxonomy.

Two authors independently reviewed and labeled the final 300 issues, considering their symptoms, root causes, and associated fixes. Agreement was assessed at the issue level: a difference in either the symptom label or the root-cause label counted as a disagreement. The resulting Cohen's kappa [29] was 0.87, indicating a high level of labeling consistency. We reconciled minor labeling disagreements through discussion and removed irrelevant issues outside the study's scope from the final dataset.

IV. WHAT ARE THE COMMON SYMPTOMS?

The symptoms documented in the 300 issue reports fall into 6 categories. Here, a symptom is the user-observed manifestation of an issue, usually described in the report's title or opening description. Table II summarizes the distribution of these categories across the seven studied libraries.

With 82 reports (27.33%), *Deployment Failure* is the most frequently observed symptom. This category covers failures during installation, dependency resolution, build orchestration, packaging, artifact selection, linking, loading, import, and startup. In one FlashAttention report, installation stopped while determining wheel build requirements because PyTorch was unavailable in the isolated build environment [30]. Users worked around this problem by installing a compatible package version or disabling build isolation.

Build Failure accounts for 73 reports (24.33%). Unlike deployment failures involving an existing package, dependency, or artifact, these cases prevent source or generated code from becoming an executable artifact. They include host, CUDA, DSL/JIT, IR, PTX, SASS, and device-code compilation failures. For example, compilation of a FlashAttention forward kernel produced invalid Multi-Level Intermediate Representation (MLIR) with an SSA dominance violation after a CUTLASS DSL upgrade [31]. Reverting the dependency avoided the failure. Diagnosis of such problems requires minimized compiler reproducers and preserved intermediate representations, alongside regression tests across compiler and GPU targets.

> **Finding 1:** More than half of the observed symptoms arise before device execution: Deployment Failure (82; 27.33%) and Build Failure (73; 24.33%). Kernel-library reliability therefore depends on packages, dependencies, compilers, and generated artifacts as well as kernel code.
>
> **Implication:** Testing should cover the full path from installation to kernel generation across supported combinations of frameworks, compilers, GPUs, and software-stack components. Preflight diagnostics should identify incompatibilities before attempting to execute an operator.

The *Crash* category contains 63 reports (21.00%). Its scope extends from API validation and pre-launch dispatch failures to unsuccessful kernel launches and faults during device execution. For example, a FlashInfer paged-KV prefill call failed before launch because no implementation supported a group size of three [32]. A FlashAttention SM100 kernel, by contrast, reached launch but failed for small head dimensions because its stage selection exceeded shared-memory capacity [33]. These cases call for explicit operator contracts, dispatch-coverage tests, and launch and runtime diagnostics.

Hang accounts for 6 reports (2.00%). These reports explicitly describe hangs or stuck processes, including coordination failures during concurrent execution and a lack of progress in kernels, builds, or compilation. For example, a CUTLASS Stream-K GEMM hung for several specific shapes because its separate-reduction path did not correctly coordinate the

```
@staticmethod
def forward(...):
    out, lse = _flash_attn_fwd(
        q, k, v, mask_mod=mask_mod,
        block_sparse_tensors=block_sparse_tensors)
    # Save sparse state for backward.
    ctx.save_for_backward(q, k, v, out, lse)
    + ctx.save_for_backward(q, k, v, out, lse,
    ↪  *(aux_tensors or ()))
    + ctx.mask_mod = mask_mod
    + ctx.block_sparse_tensors_bwd =
    ↪  block_sparse_tensors_bwd
    return out, lse

@staticmethod
def backward(ctx, dout, dlse):
    q, k, v, out, lse, *aux = ctx.saved_tensors
    dq, dk, dv = _flash_attn_bwd(
        q, k, v, out, dout, lse,
    # Restore sparse semantics in the gradient kernel.
    + mask_mod=ctx.mask_mod,
        aux_tensors=aux if aux else None,
    + block_sparse_tensors=ctx.block_sparse_tensors_bwd,
        dlse=dlse)
    return dq, dk, dv, *((None,) * 30)
```

Fig. 2. An example of Incorrect Functionality caused by losing block-sparse state between forward and backward. The fix is to save the mask and backward sparse metadata (lines 8–10) and pass them to the gradient kernel (lines 19–21).

participating work [34]. Disabling this path allowed the same workload to complete.

Not all failures interrupt execution. In *Incorrect Functionality* reports (53; 17.67%), an operation completes but produces outputs or gradients that violate its intended semantics. One FlashAttention report [35] exposed a block-sparse attention path whose forward pass matched the sparse reference while its backward pass returned finite but severely inaccurate gradients. The forward pass knew which attention blocks were enabled, but this information was not retained for the backward pass. Backward therefore computed gradients as if dense attention had run. Figure 2 shows the repair: save the mask rule and backward block list during forward, then pass both to the backward kernel. This restored agreement with an FP32 reference. The case shows that a correct forward result does not guarantee correct training; tests must also compare gradients.

Finding 2: Crash, Hang, and Incorrect Functionality account for 63 reports (21.00%), 6 reports (2.00%), and 53 reports (17.67%), respectively. Together, these 122 reports (40.67%) capture three distinct outcomes: an operation may fail during invocation or execution, stop making progress, or complete with outputs that violate its intended semantics.

Implication: Tests should check both execution outcomes and result correctness, especially at boundary conditions. In particular, they should exercise resource-sensitive dimensions and compare outputs for semantic cases such as unequal sequence lengths. A successful kernel launch alone does not establish correct behavior.

Performance Degradation accounts for 23 reports (7.67%). This category covers unexpected latency increases, reduced throughput, inefficient kernel execution, and tuning overhead. For example, a FlashAttention report documented a drop in training throughput on H100 GPUs from approximately 370 to 219 TFLOPS when the attention head dimension changed from 128 to 132 [36]. The maintainer explained that the implementations favor tensor-core-aligned dimensions and may pad unconventional dimensions to multiples such as 32, 64, or 128. The additional computation helps explain why a small change in head dimension can substantially reduce throughput.

V. What are the common root causes and fixes?

We analyzed the root causes documented in the 300 issue reports. Table III summarizes the resulting 15 categories, organized into 3 dimensions.

A. Kernel Library-side Issues

The Kernel Library dimension contains 119 cases (39.67%) across 11 categories. These cases concern computation within the kernel library, runtime decisions, and interactions between integrated components.

Code Logic Error accounts for 22 cases (7.33%). This category covers faulty predicates, branches, formulas, and special-case handling. In one DeepGEMM report, a TMA multicast heuristic used a Boolean indicating whether multicast targeted operand A as an array index [37]. After the preferred operand order was swapped, the heuristic checked operand B's legality but selected multicast on A. This mismatch could select an invalid configuration. The maintainer confirmed the logic error and corrected the array mapping so that the Boolean value and operand-legality entry used the same A/B order [38]. Repairs to this class of issues require locating the faulty control or computation logic in the kernel or its runtime selection code.

Incorrect Numerical Computation accounts for 8 cases (2.67%). These cases concern numerical transformations and aggregations whose effective semantics differ from those intended or assumed. The mechanisms include inappropriate precision or type conversion, quantization, scaling, clipping, finite-value substitution, rounding, contraction, arithmetic overflow, reduced-precision accumulation, and order-sensitive reduction. For example, a CUTLASS sparse GEMM with FP32 inputs produced values that differed substantially from a dense FP32 reference [39]. The maintainer explained that the sparse tensor-core path converted its inputs to TF32, whose shorter mantissa affected precision-sensitive values. The reporter confirmed that the outputs agreed once the dense reference also used TF32 semantics. Investigation therefore requires making the kernel's effective precision and accumulation mode explicit and comparing it with a reference that uses equivalent numerical semantics and appropriate tolerances.

Incorrect Indexing covers 4 cases (1.33%). Faulty coordinates, offsets, strides, identifiers, or index widths can select the wrong data or launch region. An xFormers attention path narrowed the number of valid key columns in the current tile to a signed 16-bit parameter [40]. The tile loop supplied the remaining key count, while online softmax used that bound to include columns in both the row maximum and exponentiation. Once the remaining span exceeded the signed 16-bit range, the predicates operated on a corrupted bound and attention returned NaN values. As Figure 3 shows, the landed repair widened the

TABLE III
CLASSIFICATION OF COMMON ROOT CAUSES IN THE SAMPLE OF 300 ISSUES.

Dimension	Category	No.	Ratio
Kernel Library	Code Logic Error	22	7.33%
	Incorrect Numerical Computation	8	2.67%
	Incorrect Indexing	4	1.33%
	Tensor Layout Mismatch	12	4.00%
	Improper Memory Access	7	2.33%
	State Mismanagement	2	0.67%
	Concurrency Error	5	1.67%
	Improper Kernel Dispatch	10	3.33%
	Resource Mismanagement	6	2.00%
	Performance Bug	17	5.67%
	API Misuse	26	8.67%
	Subtotal	119	39.67%
Compiler and Build System	Incorrect Compiler Transformation	46	15.33%
	Build Misconfiguration	35	11.67%
	Subtotal	81	27.00%
Runtime Environment	Hardware Incompatibility	24	8.00%
	Software Incompatibility	76	25.33%
	Subtotal	100	33.33%
Total		300	100.00%

```
1  // Pass the remaining valid key columns to online softmax.
2  iterative_softmax(
3      ..., p.num_keys - iter_key_start,
4      iter_key_start == 0, ...);
5
6  CUTLASS_DEVICE static void iterative_softmax(
7      ..., int8_t lane_id, int8_t thread_id, int8_t warp_id,
8      int16_t max_col,
9      + int max_col,
10     bool is_first, ...) {
11     if (accum_n < max_col)
12         max = cutlass::fast_max(max, frag[idx]);
13     // ... row reduction omitted ...
14     frag[idx] = accum_n < max_col ?
15         exp2f(frag[idx] - mi_row) : accum_t(0.0);
16 }
```

Fig. 3. An example of Incorrect Indexing caused by storing the valid-column bound in a signed 16-bit parameter (line 8), which overflows for long-context attention. The fix is to widen the parameter to `int` (line 9).

parameter to `int`; the same commit added oversized query and key regression shapes [41]. In a downstream VAE decoding workflow, the affected outputs could produce black images. Index widths must therefore cover the largest logical range before values enter tile-level predicates.

Tensor Layout Mismatch accounts for 12 cases (4.00%). These faults arise when tensor rank, stride, element width, or thread-to-data mapping does not match the representation expected by a kernel. A CUTLASS sparse GEMM kernel requires metadata in an interleaved layout named `Gemm::LayoutE`. The calling example prepared row-major metadata but passed that original tensor to the kernel rather than the tensor produced by the required layout conversion [42]. The kernel consequently interpreted the metadata using the wrong memory organization, and the correctness check observed an all-zero output. The repair converted the metadata into a separate `Gemm::LayoutE` tensor and passed that tensor to the kernel. Such cases call for validation of shape, stride, alignment, and layout contracts between caller-side data preparation and device execution. Tests should include non-contiguous, transposed, and architecture-specific representations.

Improper Memory Access comprises 7 cases (2.33%). The surrounding layout may be correct, yet invalid pointers, bounds, masks, or aliasing relationships can still violate memory-access requirements. For example, a FlashInfer sampling kernel [43] supplied the same variable as both the input and output of `FlagHeads`, a parallel primitive in NVIDIA's CUB library [44]. The primitive requires separate storage for these arguments. Aliasing them caused undefined behavior, and the sampler intermittently returned large out-of-vocabulary token identifiers instead of the only token with nonzero probability. The repair introduced a separate output variable, after which the reporter confirmed that sampling returned the expected token. Diagnosis requires checking pointer bounds, masks, alignment, and aliasing preconditions. Memory sanitizers and small test inputs with unambiguous expected outputs can help expose these faults.

With only 2 cases (0.67%), *State Mismanagement* is the smallest category in this dimension. It concerns mutable state that is not correctly initialized, reset, migrated, or preserved across invocations. In bitsandbytes, `Linear8bitLt` and `MatMul8bitLt` retained `state.idx` across batches. An index set for one batch could therefore remain active during the next. The reporter observed worse evaluation results for Llama2-13B during training than in an independent inference run after training [45]. According to the reporter's diagnosis, a later batch reused index state from an earlier batch rather than its own current state, so matrix multiplication could select the wrong data and produce incorrect results.

Concurrency Error accounts for 5 cases (1.67%). These issues involve barriers, atomics, execution ordering, progress, and producer–consumer coordination. One DeepGEMM SM90 GEMM failed in the final two-CTA cluster [46]. For the reported shape, physical CTA B8 owned the ninth and last logical tile, whereas its launched peer B9 had no tile. Each CTA-

```
// Each stage barrier needs 2 CTAs x 4 warps = 8 arrivals.
empty_barriers[i]->init(
    kNumTMAMulticast * kNumMathThreads / 32);

// Decide whether the paired multicast CTA will execute.
is_peer_cta_alive =
- kNumDBlocksPerGroup % kNumMulticast == 0 or
+ num_n_blocks % kNumMulticast == 0 or
    num_m_blocks % kNumMulticast == 0 or
    (next_block_idx ^ 1) < num_blocks;

// With multicast=2, lane 0/1 notify cluster CTA 0/1.
auto target_cta = scheduler.is_peer_cta_alive ?
    lane_idx : cute::block_rank_in_cluster();
lane_idx < kNumTMAMulticast ?
    empty_barriers[s]->arrive(target_cta) : void();
// ... consumer MMA and other pipeline work omitted ...
// B8's TMA producer waits locally before reusing stage s.
empty_barriers[s]->wait(
    (scheduler.current_iter * num_iterations + k_iter + 1)
    ↪  & 1);
```

Fig. 4. An example of a Concurrency Error caused by treating the idle peer CTA B9 as active (line 7), which routes half of B8's releases to B9 (lines 13–16) and blocks B8's local wait (lines 19–20). The fix is to use the runtime N-block count in the peer-liveness predicate (line 8).

local empty barrier nevertheless required eight consumer-warp completion arrivals. A faulty peer-liveness predicate treated B9 as active, so B8 routed four arrivals to its own barrier and four to B9's; B9 performed no consumer work and supplied no arrivals back to B8. B8's barrier therefore remained at four of eight, and its TMA producer could not pass the normal wait before reusing that shared-memory stage. As Figure 4 shows, the merged repair evaluates the alignment condition using `num_n_blocks`, detects the idle peer, and routes all eight arrivals locally [47]. Investigation of such issues requires checking barrier counts, participant liveness, and producer–consumer progress across all boundary tiles.

Improper Kernel Dispatch covers 10 cases (3.33%). An otherwise supported operation may have no valid implementation selected because a kernel variant or dispatch rule is missing or overly restrictive. In one CUTLASS report, the operation selector rejected a mixed-FP8 GEMM whose operands A and B used different FP8 formats [48]. The underlying Hopper MMA implementation supported the combination, but a static assertion required identical operand types. Template instantiation therefore stopped before the valid implementation could be selected. The restriction was soon removed.

Resource Mismanagement accounts for 6 cases (2.00%). These faults stem from incorrect calculations of storage, workspace, launch dimensions, or execution capacity. A CUTLASS Blackwell 2-SM GEMM example calculated its launch grid from the MMA tile size rather than the smaller tile assigned to each cooperative thread array (CTA) [49]. For a 1024×1024 output, the resulting grid was 4×4 instead of the required 8×4. The launch created too few CTAs to cover all output tiles and left part of the GEMM result uncomputed. The repair used the CTA tile shape to calculate the grid. Validation should check both complete coverage of the output domain and whether shared memory, workspace, and worker requirements remain within hardware and runtime limits. We assign a resource decision to this category when its calculation makes execution invalid or incomplete, rather than merely slower.

Performance Bug comprises 17 cases (5.67%). They arise from inefficient implementation choices that increase execution time or resource consumption despite producing the expected results. Typical causes include redundant computation or recompilation, unnecessary synchronization and data transfers, excessive kernel launches, and uneven work distribution. For example, a Triton attention-mask kernel [50] used values that changed between invocations as `constexpr` parameters. Each change triggered a new specialization, so JIT compilation and code generation took about 0.7 ms per call while the kernel itself ran in approximately 12 μs. Repeated compilation blocked the CPU and created execution gaps in an SGLang serving workload. A maintainer suggested checking whether compile-time arguments varied across calls, and the reporter confirmed that stabilizing them resolved the slowdown. Diagnosis requires separating kernel execution time from compilation and framework overhead, then profiling the selected path for unnecessary specialization, transfers, synchronization, or other work.

API Misuse accounts for 26 cases (8.67%). Following prior empirical work on software interface faults [51], we use this category for problems at the boundaries between wrappers, frameworks, compiler APIs, runtimes, and kernels. These problems include inconsistent types, shapes, layouts, symbols, metadata, mappings, and conversions, as well as violations of serialization, ownership, ordering, communication, or lifecycle requirements. For example, a FlashAttention change connected incompatible CUTLASS pipeline parameter types [52]. A Triton benchmarking path invoked synchronization during CUDA Graph capture and violated the rules governing that interaction [53]. Addressing these faults requires explicit interface contracts and integration tests that check both data representations and complete interaction sequences.

Finding 3: The Kernel Library cases span computation, state, dispatch, resource calculations, execution efficiency, and interfaces between integrated components. Similar symptoms can therefore require different repairs.

Implication: Testing should combine numerical comparisons, memory and resource checks, interface-contract validation, and cross-component integration tests.

B. Compiler and Build System-side Issues

Compilers and build systems turn kernel implementations into executable device code. This dimension contains 81 cases (27.00%) across the two categories in Table III.

Incorrect Compiler Transformation is the larger category, with 46 cases (15.33%). The faults occur in a library's built-in or directly integrated high-level DSL, JIT compiler, or code-generation pipeline. Frontend, IR, optimization, or lowering transformations may fail during compilation or fail to preserve valid kernel semantics. In FlashAttention, a CUTLASS DSL upgrade produced invalid MLIR with an SSA dominance violation [31]. An online-softmax fragment was used on a control-flow path where its definition did not dominate the use. IR verification consequently stopped compilation before the SM90 forward kernel could be generated. Reverting CUTLASS

DSL from 4.2.1 to 4.2.0 avoided the failure, and the maintainer later closed the report as fixed.

Build Misconfiguration accounts for 35 cases (11.67%). This category concerns build scripts, targets, generators, dependency declarations, and compiler options that prevent construction or loading, or severely impede the build process. For example, installation of a FlashAttention CUDA extension appeared to freeze during compilation [54]. Ninja [55] was not declared as a build requirement, so compilation fell back to a much slower non-parallel path rather than stopping with a clear missing-dependency message. The maintainer added Ninja to the build requirements and restored the expected parallel compilation.

Finding 4: Problems before device execution can arise from compiler transformations or build configuration. The CUTLASS DSL case stopped at IR verification, whereas the missing Ninja requirement made compilation proceed slowly enough to appear stuck.

Implication: Distinguish compilation errors and genuine non-progress from slow build progress. Preserve compiler artifacts and build logs, and verify build dependencies before concluding that compilation has stopped making progress.

C. Runtime Environment-side Issues

A kernel library needs a compatible hardware and software environment to build, load, and execute its specialized implementations. The 100 cases (33.33%) in this dimension fall into two categories, as shown in Table III.

Hardware Incompatibility accounts for 24 cases (8.00%). These issues arise when an operation requires a GPU architecture, compute capability, topology, instruction, or hardware resource that the available platform does not support. For example, a bitsandbytes user installed the CUDA extension successfully on an NVIDIA V100 but encountered an error when invoking INT8 matrix multiplication [56]. The maintainer explained that this implementation required INT8 Tensor Cores, which the V100 does not provide. Changing software versions could not supply the missing hardware capability for that kernel path. The user instead needed a compatible alternative implementation or newer hardware. GPU capability checks before dispatch, supported fallbacks where available, and diagnostics that name the missing architectural feature can make these failures easier to understand and address.

Software Incompatibility is the largest individual root-cause category, with 76 cases (25.33%). It covers incompatibilities among the library's dependencies and execution environment, including GPU drivers, CUDA/ROCm runtimes, deep learning frameworks, operating systems, ABIs, and their version requirements. In one FlashAttention report, a call failed at runtime although CUDA 11.6 was available on the host [57]. The installed PyTorch package had been compiled against CUDA 11.3, below the library's requirement. The same input worked after the reporter installed a PyTorch build compiled for CUDA 11.6. This comparison isolated the failure to the software combination rather than the GPU or attention kernel. Diagnosis should record both installed and build-time versions, check compatibility across the complete stack, and reproduce the problem with a documented supported combination.

Finding 5: The Runtime Environment dimension accounts for 100 reports (33.33%). Software Incompatibility is the largest individual category, with 76 reports (25.33%).

Implication: Compatibility checks should consider package, framework, compiler, CUDA, and driver versions together with the GPU architecture. Updating a single component does not establish that the resulting combination is supported.

VI. Current testing and debugging practices

The selected reports document not only failures and fixes but also the tests and diagnostic steps used to investigate them. We summarize these accounts to examine how developers investigate and resolve GPU kernel library issues. Our evidence is limited to what appears in the reports and their discussions; it does not establish whether developers also used practices that went unrecorded.

A. Testing Practices

GPU kernel tests must check both the computed results and the implementation's behavior under supported execution conditions. Drawing on the reported cases and guidance from NVIDIA and PyTorch [58]–[60], we summarize five complementary testing practices.

Compatibility and Integration Testing. Developers record the hardware and software stack and use representative supported inputs to check interactions among frameworks, wrappers, compilers, and kernels. In one CUTLASS report, a Turing Tensor Core test failed with CUDA 10.1 because the operation required CUDA 10.2 or later [61]. Such checks can expose platform, version, and interface incompatibilities before they surface as opaque kernel errors.

Differential Testing. The optimized kernel and a simpler reference receive the same input, and their outputs are compared under consistent settings and an appropriate tolerance. The reference itself also requires scrutiny. In one FlashAttention report, gradients disagreed with an FP16 reference but matched the same calculation in FP32 [62]. The discrepancy arose from rounding error in the reference rather than a defect in the optimized kernel. Differential testing can reveal silent incorrectness, but further diagnosis is needed to establish the source of a mismatch.

Boundary Value Analysis. Developers vary one dimension, sequence-length relationship, alignment, or layout at a time while keeping other settings fixed. In a FlashAttention issue, a sweep showed failures at head dimensions 8 and 16 but not 32, which helped isolate an incorrect shared-memory calculation [33]. A separate report required unequal query and key lengths to reveal a masking error [63]. Tests restricted to common shapes can leave such faults undetected.

Dynamic Analysis. NVIDIA Compute Sanitizer [59] provides checks for memory access, races, initialization, and synchro-

nization. In a FlashInfer report involving a 48-rank CuTe DSL [64] GEMM, it detected an illegal memory access [65].

Performance Regression Testing. Developers compare a fixed workload against a baseline after warm-up and measure framework, compilation, and kernel time separately [60]. A Triton report illustrates the importance of this separation: each invocation spent about 0.7 ms in repeated JIT compilation, while the kernel itself took approximately 12 μs [50]. These measurements placed the slowdown outside GPU execution. NVIDIA Nsight Systems [66] and Nsight Compute [67] can further help identify the critical path and kernel bottlenecks.

B. Debugging Practices

Diagnosis typically begins with the evidence attached to a report: error messages, stack traces, generated compiler output, and details of the hardware and software environment. Developers then attempt to reproduce the failure with the supplied input and inspect the relevant wrapper, compiler, or kernel code. For straightforward cases, the remedy may be a code correction, a configuration change, or a switch to a compatible software version.

More difficult cases require a small reproducer that retains the conditions responsible for the failure. A compact FlashInfer sampling script, for example, returned invalid token identifiers, but the maintainer initially could not reproduce the behavior. A comparison of CUDA environments revealed that only the failing environment selected a CUB [44] operation subject to an aliasing restriction. Locating the fault therefore required both the reproducing input and the relevant software version.

Specialized diagnostics help narrow the investigation to a particular layer. NVIDIA Compute Sanitizer can identify memory and synchronization faults, while compiler intermediate representation (IR) and version comparisons provide evidence for failures before execution. In one FlashAttention report, an IR verification error appeared after a CUTLASS DSL upgrade and disappeared when the dependency was reverted. The generated IR localized the failure to an SSA dominance violation and supplied evidence for an upstream report. The issue did not, however, identify the exact repair commit.

Some failures remain difficult to reproduce because they depend on a particular shape, layout, backend, hardware target, or software combination. Developers investigate these dependencies by changing one factor at a time, rerunning the reproducer, and comparing outcomes. An issue first observed in a model or serving system may require investigation across several library boundaries. A proposed fix should be checked against the original reproducer and its expected result. The disappearance of an error after a workload change does not, by itself, establish that the underlying fault has been repaired.

VII. Discussion

A. Threats to Validity

Threats to Internal Validity. Classifying symptoms and root causes and interpreting fixes require judgment, particularly when an issue is complex or its supporting evidence is incomplete. Issue descriptions, discussions, and linked fixes inform these judgments but do not eliminate subjectivity. To limit this threat, we established explicit coding rules and retained the evidence supporting each classification. Two reviewers independently examined all 300 cases. Their agreement, measured with Cohen's kappa [29], was 0.87, indicating a high level of consistency. We further discussed disagreements collectively and sought consensus before making critical classification decisions.

Threats to External Validity. The selection of libraries and issue reports limits how broadly our findings can be generalized. Our subjects comprise seven open-source GPU kernel libraries that cover attention, GPU programming, GEMM, inference, and quantization. They include both research-originated projects and reusable industrial libraries. Their use extends across training ecosystems such as Megatron-LM, PyTorch, Transformers, and DeepSpeed, as well as serving systems like vLLM and SGLang. These connections support the relevance of the observed issues beyond a single repository. They do not, however, establish that the findings apply equally to other GPU kernel projects, proprietary kernels, or complete training and serving systems. In addition, replacing reports without identifiable causes may favor well-documented cases. To mitigate this threat, we therefore distinguish repository-specific observations from aggregate results and avoid generalizing findings that may be specific to the seven studied libraries to GPU kernel libraries as a whole.

B. Generality of Our Study

Our findings can be relevant to other GPU kernel projects that share the technical characteristics of the seven studied libraries. The studied libraries connect Python or C++ interfaces with CUDA, Triton, or generated device code through similar heterogeneous programming stacks. They also implement common LLM operators—attention, GEMM, quantization, and low-precision computation—with related constraints on tensor shapes, layouts, numerical precision, dispatch, and hardware resources. Their deployment places them within comparable framework, compiler, driver, and GPU environments, with integrations into both training and serving systems. These commonalities suggest that similar failure mechanisms may recur elsewhere and that the testing and debugging lessons drawn from our cases may transfer to other projects.

Evidence from related systems provides additional support for this interpretation. Empirical studies of deep learning software have documented problems involving configuration, APIs, dependencies, and execution environments [9], [10], [15], [16], [24]. Studies of deep learning compilers likewise report defects in optimization and code generation [8]. GPU verification and testing research addresses races, barrier divergence, memory-access errors, and compiler defects [68]–[72]. The overlap between these independent lines of work and our taxonomy provides further evidence that the identified mechanisms are not unique to the seven repositories examined here.

C. Future Research Directions

The findings suggest several opportunities to improve tool support and library design. The following directions could help

developers detect incompatibilities earlier, diagnose failures more precisely, and reproduce problems reliably.

Tool Support:

Compatibility Checker. Runtime Environment accounts for 100 cases (33.33%), while Deployment Failure is the largest symptom category, with 82 cases (27.33%). These findings motivate a compatibility checker that examines the GPU, driver, runtime, compiler, framework, package, and binary ABI before a build or execution begins. The checker should distinguish installed software versions from those used to build frameworks and binary extensions, since these may differ. When a requirement is violated, it could identify the conflicting components, explain the incompatibility, and recommend a supported combination. Such diagnostics would give developers a concrete starting point instead of leaving them to interpret a generic build or GPU kernel error.

Numerical Debugging Tools. Incorrect Functionality accounts for 53 cases (17.67%), with causes that include Code Logic Error, Incorrect Numerical Computation, Incorrect Indexing, Tensor Layout Mismatch, and Improper Memory Access. Numerical debugging tools could compare kernel and reference values at intermediate stages and track the associated shapes, data types, accumulation modes, masks, and tolerances. TensorFuzz [73] and Herbie [74] illustrate techniques for detecting anomalous numerical behavior and improving floating-point expressions. Adapting such techniques to specialized GPU kernels requires careful treatment of their numerical contracts.

Compiler Debugging Tools. Incorrect Compiler Transformation accounts for 46 cases (15.33%). A compiler debugging tool could reduce the triggering kernel and preserve its source, generated IR, compiler version, compilation options, and target architecture for differential and regression analysis. IR snapshots could help identify the transformation at which compilation fails or valid kernel semantics are lost. Compiler-testing systems such as CUDAsmith [71] and NNSmith [75] provide techniques for generating programs that expose compiler defects. Combining test generation with failure reduction and artifact preservation could make reports easier to investigate and turn confirmed defects into focused regression tests.

Library Enhancements:

Contract-aware Validation. Pre-launch API failures and issues involving Improper Kernel Dispatch, Tensor Layout Mismatch, Improper Memory Access, and Resource Mismanagement often reflect implicit kernel requirements that are violated or calculated incorrectly. Libraries could express contracts for shapes, data types, layouts, alignment, aliasing, supported in-place operations, and shared-memory requirements as executable checks in their wrappers and dispatch layers. These checks should run before kernel launch and report both the expected and supplied values. Where a requirement depends on the selected implementation or GPU architecture, the diagnostic should identify that dependency explicitly.

Record and Replay. Many kernel failures depend on a particular input, selected implementation, compiler artifact, software stack, or GPU architecture. Reconstructing this combination from an issue discussion can be difficult, especially after dependencies or implementations have changed. Libraries could record the operation, tensor metadata, dispatch decision, software versions, target architecture, and generated IR needed to replay the failure [76]. Metadata alone may be insufficient for value-dependent failures, so replay support should also preserve the triggering input or a reproducible procedure for generating it. Once the cause is confirmed, the recorded case could become a regression test that recreates the relevant configuration and checks the expected result or behavior.

VIII. Related Work

Attention, GEMM, and quantization recur across large-scale LLM workloads, making specialized GPU kernels valuable to many models and systems. TVM [77], Ansor [78], Triton [3], and FlashAttention [1] reduce memory traffic and improve throughput through techniques such as fusion, tiling, low-precision computation, and hardware-specific instructions. These optimizations also impose complex requirements on compilation, data layout, dispatch, and compatibility.

Empirical research has examined defects at several levels of the AI software stack. Studies of TensorFlow and other deep learning frameworks document API, configuration, data, and numerical problems, as well as silent behavioral bugs [9], [15], [16], [23], [24], [79]. Other work addresses performance bugs in TensorFlow and PyTorch [80], deployment faults in deep learning applications [27], production job failures [81], and distributed-training challenges [28]. Gao et al. [11] analyzed issues in DeepSpeed, Megatron-LM, and Transformers, with attention to their symptoms, root causes, fixes, and testing and debugging practices. Shen et al. [8] investigated defects in deep learning compilers.

Complementary research on GPU verification and compiler testing targets races, barrier divergence, memory faults, and compiler defects through tools such as GPUVerify [68], GKLEE [69], CURD [70], GPUMC [72], CUDAsmith [71], and NNSmith [75]. TensorFuzz [73] and Herbie [74] address anomalous floating-point behavior and numerical accuracy.

The empirical literature largely concerns applications, frameworks, training platforms, and compilers, while existing testing techniques focus on particular fault classes. Our study instead examines reported symptoms, root causes, fixes, and engineering practices across the lifecycle of seven GPU kernel libraries. The analysis spans the Kernel Library, Compiler and Build System, and Runtime Environment dimensions.

IX. Conclusion

Our empirical analysis of 300 issues from seven GPU kernel libraries identified 6 symptom categories and 15 root-cause categories across 3 dimensions. We also documented typical fixes and reported testing and debugging practices, and outlined research directions for tool support and library enhancements. These findings provide a clearer understanding of GPU kernel library issues and practical guidance for developing more reliable and efficient LLM systems.

REFERENCES

[1] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35. Neural Information Processing Systems Foundation, 2022, pp. 16 344–16 359. [Online]. Available: https://doi.org/10.52202/068431-1189

[2] FlashInfer contributors, "FlashInfer," https://github.com/flashinfer-ai/flashinfer, 2026, accessed: 2026-08-10.

[3] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. ACM, 2019, pp. 10–19. [Online]. Available: https://doi.org/10.1145/3315508.3329973

[4] NVIDIA, "CUTLASS," https://github.com/NVIDIA/cutlass, 2026, accessed: 2026-08-10.

[5] DeepSeek-AI, "DeepGEMM," https://github.com/deepseek-ai/DeepGEMM, 2026, accessed: 2026-08-10.

[6] Meta AI, "xFormers," https://github.com/facebookresearch/xformers, 2026, accessed: 2026-08-10.

[7] bitsandbytes contributors, "bitsandbytes," https://github.com/bitsandbytes-foundation/bitsandbytes, 2026, accessed: 2026-08-10.

[8] Q. Shen, H. Ma, J. Chen, Y. Tian, S.-C. Cheung, and X. Chen, "A comprehensive study of deep learning compiler bugs," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 968–980.

[9] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. ACM, 2020, pp. 1110–1121. [Online]. Available: https://doi.org/10.1145/3377811.3380395

[10] Y. Gao, X. Shi, H. Lin, H. Zhang, H. Wu, R. Li, and M. Yang, "An empirical study on quality issues of deep learning platform," in *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2023, pp. 455–466.

[11] Y. Gao, R. Lu, H. Lin, and Y. Chen, "An empirical study of issues in large language model training systems," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '25. Association for Computing Machinery, 2025, pp. 122–133.

[12] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," *CoRR*, vol. abs/1909.08053, 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1909.08053

[13] vLLM contributors, "vLLM," https://github.com/vllm-project/vllm, 2026, accessed: 2026-08-10.

[14] SGLang contributors, "SGLang," https://github.com/sgl-project/sglang, 2026, accessed: 2026-08-10.

[15] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, and L. Zhang, "An empirical study on TensorFlow program bugs," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 129–140. [Online]. Available: https://doi.org/10.1145/3213846.3213866

[16] T. Zhang, C. Gao, L. Ma, M. Lyu, and M. Kim, "An empirical study of common challenges in developing deep learning applications," in *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*, 2019, pp. 104–115.

[17] J. Cao, B. Chen, C. Sun, L. Hu, S. Wu, and X. Peng, "Understanding performance problems in deep learning systems," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 357–369. [Online]. Available: https://doi.org/10.1145/3540250.3549123

[18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[19] Hugging Face, "Transformers," https://github.com/huggingface/transformers, 2024.

[20] A. Di Franco, H. Guo, and C. Rubio-González, "A comprehensive study of real-world numerical bug characteristics," in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 509–519.

[21] GitHub, "Github rest api documentation," https://docs.github.com/en/rest, accessed September 15, 2026.

[22] L. Tan, C. Liu, Z. Li, X. Wang, Y. Zhou, and C. Zhai, "Bug characteristics in open source software," *Empirical Softw. Engg.*, vol. 19, no. 6, p. 1665–1705, Dec. 2014. [Online]. Available: https://doi.org/10.1007/s10664-013-9258-8

[23] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, "A comprehensive study on deep learning bug characteristics," in *Proceedings of the 27th ACM Joint Meeting on ESEC/FSE*, 2019, pp. 510–520.

[24] L. Jia, H. Zhong, X. Wang, L. Huang, and X. Lu, "An empirical study on bugs inside tensorflow," in *Database Systems for Advanced Applications: 25th International Conference, DASFAA 2020*, 2020, pp. 604–620.

[25] ——, "The symptoms, causes, and repairs of bugs inside a deep learning library," *Journal of Systems and Software*, vol. 177, p. 110935, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121221000327

[26] J. Chen, Y. Liang, Q. Shen, J. Jiang, and S. Li, "Toward understanding deep learning framework bugs," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, Sep. 2023. [Online]. Available: https://doi.org/10.1145/3587155

[27] Z. Chen, H. Yao, Y. Lou, Y. Cao, Y. Liu, H. Wang, and X. Liu, "An empirical study on deployment faults of deep learning based mobile applications," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 674–685.

[28] X. Liu, D. Gu, Z. Chen, J. Wen, Z. Zhang, Y. Ma, H. Wang, and X. Jin, "Rise of distributed deep learning training in the big model era: From a software engineering perspective," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, Sep. 2023. [Online]. Available: https://doi.org/10.1145/3597204

[29] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[30] FlashAttention, "Error: Install pip install flash-attn," 2023. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/258

[31] ——, "Cute-dsl sm90 fwd produces error after bumping to nvidia-cutlass-dsl-4.2.1," 2025. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/1918

[32] FlashInfer, "[bug report] batchprefillwithpagedkvcachepytorchwrapper failed to dispatch group_size 3," 2024. [Online]. Available: https://github.com/flashinfer-ai/flashinfer/issues/258

[33] FlashAttention, "[fa4, cute, sm100] _flash_attn_fwd fails at launch for head_dim $<$ 32 on b200," 2026. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/2591

[34] NVIDIA, "[bug] tma cooperative gemm with stream-k scheduler hangs for specific gemm shapes," 2024. [Online]. Available: https://github.com/NVIDIA/cutlass/issues/1801

[35] FlashAttention, "[cute/sm90] block-sparse backward returns silently-wrong gradients for head_dim=64 (forward is exact)," 2026. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/2623

[36] ——, "The impact of setting head dim to an unconventional value on the algorithm's running speed," 2024. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/1141

[37] DeepSeek, "[bug]tma multicast code issue," 2025. [Online]. Available: https://github.com/deepseek-ai/DeepGEMM/issues/190

[38] ——, "Fix multicast bug and optimize masked gemm," 2025. [Online]. Available: https://github.com/deepseek-ai/DeepGEMM/pull/193

[39] NVIDIA, "[bug] sparse gemm seems to produce wrong results for f32 datatype and specific inputs," 2023. [Online]. Available: https://github.com/NVIDIA/cutlass/issues/1270

[40] Meta, "[0.0.18] memory_efficient_attention nans when seqlen $>$ 32768," 2023. [Online]. Available: https://github.com/facebookresearch/xformers/issues/719

[41] ——, "[cutlassf] fix k sequence length overflow," 2023. [Online]. Available: https://github.com/facebookresearch/xformers/commit/68dce697013d3078315174840106bc9a81477d82

[42] NVIDIA, "15_ampere_sparse_tensorop_gemm fails," 2021. [Online]. Available: https://github.com/NVIDIA/cutlass/issues/225

[43] FlashInfer, "top_k_top_p_sampling_from_probs occasionally outputs invalid token ids," 2024. [Online]. Available: https://github.com/flashinfer-ai/flashinfer/issues/384

[44] NVIDIA Corporation, "CUB: Cooperative primitives for CUDA C++," https://github.com/nvidia/cub, 2024.

[45] bitsandbytes, "state.idx never gets reset in linear8bitlt and matmul8bitlt," 2023. [Online]. Available: https://github.com/bitsandbytes-foundation/bitsandbytes/issues/767

[46] DeepSeek, "[bug] unspecified launch failure of normal gemm when n=16," 2025. [Online]. Available: https://github.com/deepseek-ai/DeepGEMM/issues/147

[47] ——, "Fix sm90 gemm," 2025. [Online]. Available: https://github.com/deepseek-ai/DeepGEMM/pull/149

[48] NVIDIA, "[bug] rs_op_selector does not allow to use mixed fp8 types for a and b," 2023. [Online]. Available: https://github.com/NVIDIA/cutlass/issues/1268

[49] ——, "[bug] kernel launch griddim was miscalculated in examples/cute/tutorial/blackwell/04_mma_tma_2sm_sm100.cu," 2025. [Online]. Available: https://github.com/NVIDIA/cutlass/issues/2494

[50] Triton, "Triton compile takes extremely long at runtime," 2025. [Online]. Available: https://github.com/triton-lang/triton/issues/6480

[51] D. E. Perry and W. M. Evangelist, "An empirical study of software interface faults," in *International Symposium on New Directions in Computing*. IEEE Computer Society, 1985, pp. 32–38. [Online]. Available: https://users.ece.utexas.edu/~perry/work/papers/isnd.pdf

[52] FlashAttention, "Fa3 build error with large-dim changes," 2025. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/1476

[53] Triton, "cuda graph stream invalidate by sync call inside do_bench_cudagraph," 2026. [Online]. Available: https://github.com/triton-lang/triton/issues/10596

[54] FlashAttention, "The installation process always freezes after execution," 2023. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/215

[55] Ninja, "Ninja," https://ninja-build.org, 2011, ninja is a small build system with a focus on speed.

[56] bitsandbytes, "Error: Your gpu does not support int8 matmul! nvidia v100," 2023. [Online]. Available: https://github.com/bitsandbytes-foundation/bitsandbytes/issues/444

[57] FlashAttention, "Indexerror about flash_attn_unpadded_qkvpacked_func," 2023. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/217

[58] NVIDIA Corporation, "CUDA C++ Best Practices Guide," https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/, 2026, accessed: September 17, 2026.

[59] ——, "Compute Sanitizer User Manual," https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/, 2026, accessed: September 17, 2026.

[60] PyTorch Contributors, "PyTorch Benchmark," https://docs.pytorch.org/tutorials/recipes/recipes/benchmark.html, 2025, accessed: September 17, 2026.

[61] NVIDIA, "08_turing_tensorop_gemm test," 2020. [Online]. Available: https://github.com/NVIDIA/cutlass/issues/80

[62] FlashAttention, "There seems to be errors in a small number of elements of dk in flash-attention-2," 2023. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/385

[63] ——, "causal argument when q_sequence is not cu_seqlens_q isn't aligned with cu_seqlens_k," 2023. [Online]. Available: https://github.com/Dao-AILab/flash-attention/issues/282

[64] NVIDIA, "CuTe DSL introduction," 2026, accessed: 2026-10-23. [Online]. Available: https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl_general/dsl_introduction.html

[65] FlashInfer, "Cutedsl gemm illegal access on 48 ranks," 2025. [Online]. Available: https://github.com/flashinfer-ai/flashinfer/issues/1618

[66] NVIDIA Corporation, "NVIDIA Nsight Systems User Guide," https://docs.nvidia.com/nsight-systems/UserGuide/, 2026, accessed: September 17, 2026.

[67] ——, "NVIDIA Nsight Compute User Guide," https://docs.nvidia.com/nsight-compute/NsightCompute/, 2026, accessed: September 17, 2026.

[68] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, and P. Thomson, "GPUVerify: A verifier for GPU kernels," in *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications*. ACM, 2012, pp. 113–132. [Online]. Available: https://doi.org/10.1145/2384616.2384625

[69] G. Li, P. Li, G. Sawaya, G. Gopalakrishnan, I. Ghosh, and S. P. Rajan, "GKLEE: Concolic verification and test generation for GPUs," in *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. ACM, 2012, pp. 215–224. [Online]. Available: https://doi.org/10.1145/2145816.2145844

[70] Y. Peng, V. Grover, and J. Devietti, "CURD: A dynamic CUDA race detector," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2018, pp. 390–403. [Online]. Available: https://doi.org/10.1145/3192366.3192368

[71] B. Jiang, X. Wang, W. K. Chan, T. H. Tse, N. Li, Y. Yin, and Z. Zhang, "CUDAsmith: A fuzzer for CUDA compilers," in *2020 IEEE 44th Annual Computers, Software, and Applications Conference*. IEEE, 2020, pp. 861–871. [Online]. Available: https://doi.org/10.1109/COMPSAC48688.2020.0-156

[72] S. Chakraborty, S. Krishna, A. Pavlogiannis, and O. Tuppe, "GPUMC: A stateless model checker for GPU weak memory concurrency," in *Computer Aided Verification*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland, 2025, pp. 321–346. [Online]. Available: https://doi.org/10.1007/978-3-031-98682-6_17

[73] A. Odena, C. Olsson, D. Andersen, and I. Goodfellow, "TensorFuzz: Debugging neural networks with coverage-guided fuzzing," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 4901–4911. [Online]. Available: https://proceedings.mlr.press/v97/odena19a.html

[74] P. Panchekha, A. Sanchez-Stern, J. R. Wilcox, and Z. Tatlock, "Automatically improving accuracy for floating point expressions," in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2015, pp. 1–11. [Online]. Available: https://doi.org/10.1145/2737924.2737959

[75] J. Liu, J. Lin, F. Ruffy, C. Tan, J. Li, A. Panda, and L. Zhang, "NNSmith: Generating diverse and valid test cases for deep learning compilers," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, 2023, pp. 530–543. [Online]. Available: https://doi.org/10.1145/3575693.3575707

[76] Z. Guo, X. Wang, J. Tang, X. Liu, Z. Xu, M. Wu, M. F. Kaashoek, and Z. Zhang, "R2: an application-level kernel for record and replay," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI '08. USA: USENIX Association, 2008, pp. 193–208.

[77] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated End-to-End optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 578–594. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/chen

[78] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating High-Performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 2020, pp. 863–879. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zheng

[79] F. Tambon, A. Nikanjam, L. An, F. Khomh, and G. Antoniol, "Silent bugs in deep learning frameworks: An empirical study of keras and tensorflow," *ArXiv*, vol. abs/2112.13314, 2021.

[80] T. Makkouk, D. J. Kim, and T.-H. P. Chen, "An empirical study on performance bugs in deep learning frameworks," in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2022, pp. 35–46.

[81] R. Zhang, W. Xiao, H. Zhang, Y. Liu, H. Lin, and M. Yang, "An empirical study on program failures of deep learning jobs," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ser. ICSE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1159–1170. [Online]. Available: https://doi.org/10.1145/3377811.3380362